

TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI
BỘ MÔN KHOA HỌC MÁY TÍNH

THIẾT KẾ VÀ ĐÁNH GIÁ THUẬT TOÁN

Chương 4

Các phương pháp thử sai (*Trial and Error Methods*)

Phạm Hồng Phong
phongph@huce.edu.vn

Nội dung

- 1. Phương pháp vét cạn**
- 2. Phương pháp quay lui**
- 3. Phương pháp nhánh cận**

§ 1. Phương pháp vét cạn

- **Ý tưởng**

- Duyệt tất cả các phương án có thể

- **Mô hình**

Giả sử $D = \{(x_1, x_2, \dots, x_n)\}$ là không gian nghiệm của bài toán (tập khả năng)

Trong đó $x_i \in D_i$, với $D_i = \{d_{ij} \mid j = 1 \dots m_i\}$ là tập có m_i phần tử

Quy tắc xác định lời giải bài toán:

$$f: D \rightarrow \{true, false\}$$

Nghiệm của bài toán: $x = (x_1, x_2, \dots, x_n)$ sao cho $f(x) = true$

§ 1. Phương pháp vét cạn

- **Lược đồ phương pháp**

VetCan (D) $\equiv$

For ($x_1 = d_{11} \dots d_{1m1}$)

For ($x_2 = d_{21} \dots d_{2m2}$)

For ($x_n = d_{n1} \dots d_{nmn}$)

if ($f(x_1, x_2, \dots, x_n)$)

 <Chấp nhận nghiệm $x = (x_1, x_2, \dots, x_n)$ >;

End.

§ 1. Phương pháp vét cạn

- Ví dụ

- Bài toán gà, chó

Vừa gà vừa chó; Bó lại cho tròn;

Ba mươi sáu con; Một trăm chân chẵn

Hỏi số gà, số chó?

- Phân tích:

Gọi x, y là số gà, chó: $x \in D_1 = \{0, 1, \dots, 36\}, y \in D_2 = \{0, 1, \dots, 36\}$

Không gian nghiệm của bài toán: $D = \{(x, y)\}$

Quy tắc xác định lời giải: $f(x, y) = (x + y = 36) \text{ and } (2x + 4y = 100)$

Nghiệm của bài toán $x = (x, y)$ sao cho $f(x, y) = \text{true}$

§ 1. Phương pháp vét cạn

- Ví dụ: *Bài toán gà chó*

- Lược đồ thuật bài toán:

GaCho $\equiv$

For (x=0 .. 36)

For (y=0 .. 36)

if ((x+y) = 36 & (2x + 4y) = 100)

<In ra kết quả (x,y)>;

End.

- Độ phức tạp thuật toán: $O(n^2)$, $n=36$ – phụ thuộc không gian bài toán
- Cải tiến:
 - Giảm không gian nghiệm bài toán: $D=\{(x)\}$, $x \in D_1=\{0,1,\dots,25\}$
 - Quy tắc xác định nghiệm: $f(x)=(4x+2(36-x)=100)$
 - Độ phức tạp: $O(n)$, $n=25$

§ 1. Phương pháp vét cạn

- **Các bài toán ứng dụng:**

- Các bài toán tìm kiếm:
 - Tìm phần tử trong dãy
 - Tìm chuỗi ký tự trong xâu (string matching)
 - Tìm cặp điểm gần nhất (closest pair)
 - ...
- Một số bài toán xác định phương án nghiệm từ nhiều phương án khả thi
 - Xây dựng chu trình Euler, Hamilton của đồ thị
 - Bài toán người bán hàng (travelling salesman)
 - Bài toán xếp balo (knapsack)
 - ...

=> khả thi với lượng dữ liệu nhỏ

§ 2. Phương pháp quay lui

Trong nhiều bài toán, việc tìm nghiệm có thể quy về tìm cấu hình
($x_1, x_2, \dots, x_n, \dots$)

- Độ dài của cấu hình có thể được xác định trước hoặc không
- Cấu hình phải thỏa mãn yêu cầu của bài toán
- Thành phần x_i được chọn từ tập hữu hạn A_i
- Yêu cầu của bài toán có thể là: tìm một nghiệm, tìm tất cả các nghiệm hoặc đếm số nghiệm

§ 2. Phương pháp quay lui

Các vấn đề quan trọng khi sử dụng quay lui:

- Xác định được biểu diễn cấu hình
- Xác định tập S_i gồm những ứng viên cho x_i
- Xác định điều kiện để cấu hình là nghiệm

§ 2. Phương pháp quay lui

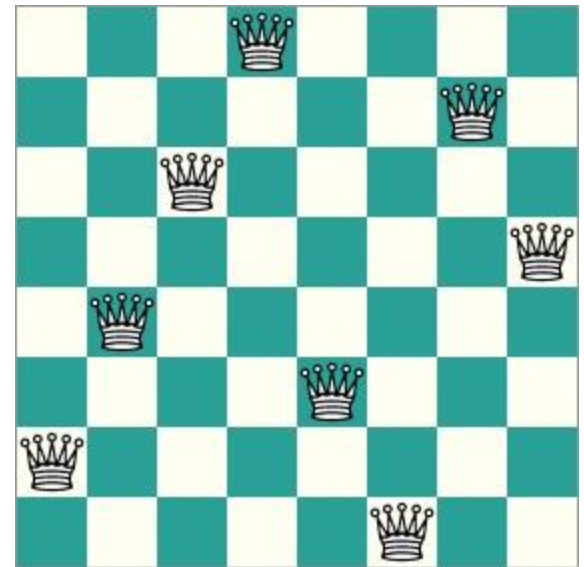
Ví dụ: Bài toán 8 quân hậu

Cần đặt 8 quân hậu vào một bàn cờ vua 8x8 sao cho chúng không tấn công nhau?

- Các quân hậu phải nằm trên các hàng khác nhau: quân hậu i là quân hậu nằm ở hàng i
- Nghiệm của bài toán là cấu hình $(x_1, x_2, \dots, x_8)$ với x_i là cột mà quân hậu i đứng

$$(x_i \in A_i = \{1, \dots, 8\})$$

- $x_i \neq x_j$ với mọi $i, j = 1, \dots, 8$
- Hai ô (i, x_i) và (j, x_j) không cùng nằm trên một đường chéo với mọi $i, j = 1, \dots, 8$



§ 2. Phương pháp quay lui

- **Ý tưởng**

- Quay lui (Back tracking): Theo nguyên tắc vét cạn, nhưng chỉ xét những trường hợp khả quan.
- Dùng để giải bài toán liệt kê các cấu hình:
 - Mỗi cấu hình được xây dựng bằng cách xác định từng phần tử
 - Mỗi phần tử được chọn bằng cách thử các khả năng ***có thể***.
- Tại mỗi bước, nếu có một lựa chọn được chấp nhận thì ghi nhận lại lựa chọn này và tiến hành các bước thử tiếp theo. Còn ngược lại không có lựa chọn nào thích hợp thì quay lại bước trước $\Rightarrow$ ***quay lui***.

§ 2. Phương pháp quay lui

- **Lược đồ dạng đệ quy**

```
Backtrack(i)  $\equiv$  //Sinh thành phần thứ i của cấu hình  
    <Xác định Si>;  
    for (xi  $\in$  Si)  
        <ghi nhận thành phần thứ i>;  
        if(tìm thấy nghiệm) then <đưa ra nghiệm>  
        else Backtrack (i + 1);  
        <loại thành phần thứ i>;  
    endfor  
End.
```

§ 2. Phương pháp quay lui

- **Ví dụ 1:**

- Bài toán: Liệt kê dãy nhị phân độ dài n

- Phân tích:

- Dãy nhị phân $(x_1x_2\cdots x_n)$ trong đó $x_i \in \{0,1\}$
- Dùng giải thuật $\text{Backtrack}(i)$ để sinh giá trị x_i
- Nếu $i=n$ thì in giá trị nghiệm, ngược lại sinh tiếp x_{i+1} bằng $\text{Backtrack}(i+1)$

- Lược đồ:

$\text{Backtrack}(i) \equiv$

for ($v=0..1$) *//v nhận giá trị 0 hoặc 1*

$x_i = v;$

if ($i= n$) $\text{printResult}(x);$

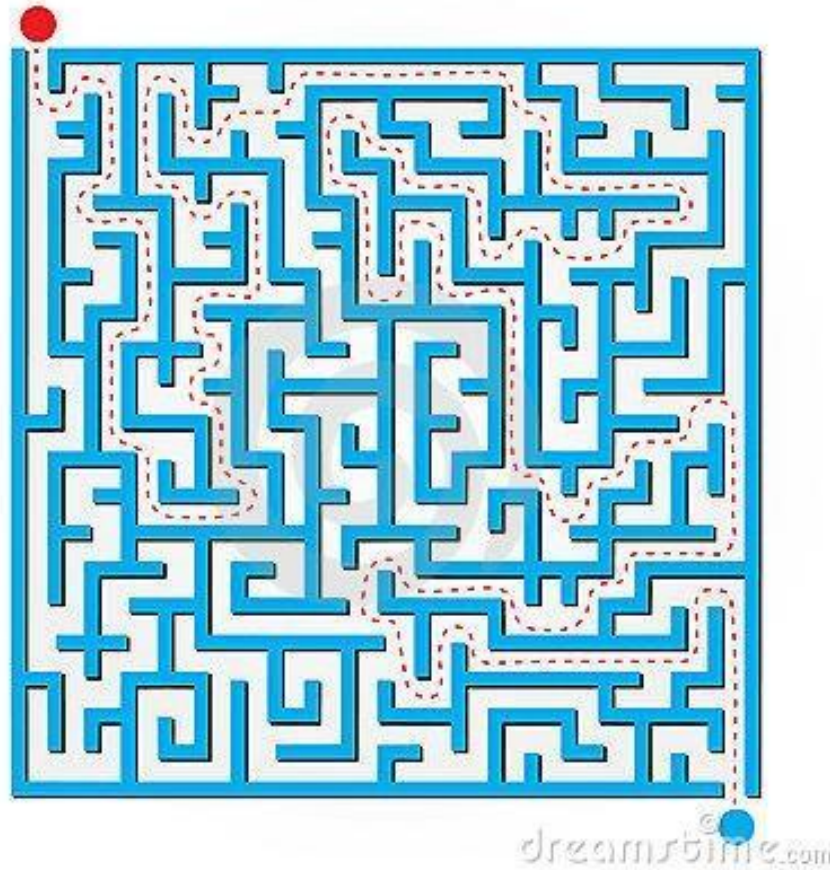
else $\text{Backtrack}(i+1);$

End. endfor;

Lời gọi ban đầu $\text{Backtrack}(1)$

§ 2. Phương pháp quay lui

- **Ví dụ 2:**
 - Bài toán tìm đường đi trong mê cung (maze):



§ 2. Phương pháp quay lui

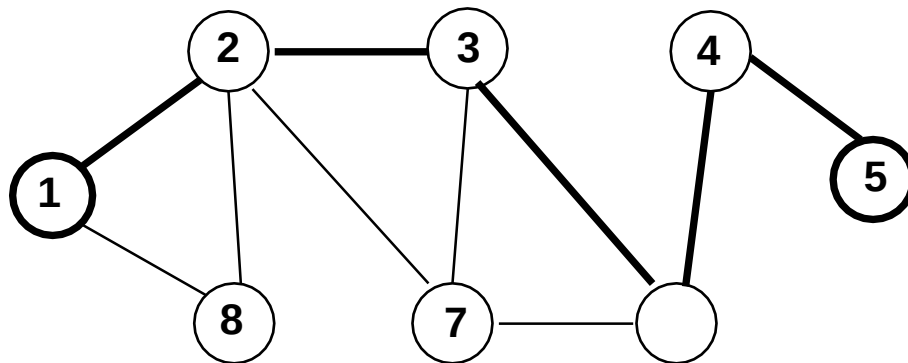
- **Ví dụ 2: Tìm đường trong mê cung**

- Phân tích: Xét mê cung như một đồ thị vô hướng $G=(V,E)$

- Mỗi phòng của mê cung là một nút của đồ thị
- Mỗi hành lang giữa các phòng là một cạnh nối giữa các nút của đồ thị
- Đỉnh xuất phát: $S \in V$, đỉnh kết thúc $F \in V$
- Đường đi tìm được là dãy $S=x_1,x_2,\dots,x_k=F$ với $x_i \in V, (x_i,x_{i+1}) \in E$
- Dùng giải thuật Backtrack(i) để sinh giá trị x_i
 - ✳ Đỉnh v được chấp nhận cho x_i nếu v chưa được đi qua, có cạnh (x_{i-1},v)
 - ✳ Nếu $x_i=F$ thì kết thúc, in đường đi tìm được
 - ✳ Ngược lại sinh tiếp x_{i+1} bằng Backtrack($i+1$)
- Lời gọi ban đầu
 - ✳ $x_1=S$
 - ✳ Backtrack(2)

§ 2. Phương pháp quay lui

- Ví dụ 2: Tìm đường trong mê cung



- Đỉnh xuất phát: $S=1 \in V$, đỉnh kết thúc $F=5 \in V$
- Đường đi tìm được là dãy $S=1-2-3-6-4-5=F$

§ 2. Phương pháp quay lui

- **Ví dụ 2: Tìm đường trong mê cung**

- Mô tả dữ liệu

- Đánh chỉ số các đỉnh của đồ thị từ $1..n$
- Biểu diễn đồ thị G bằng ma trận kề $M = (m_{ij})$ cỡ $n \times n$
 $m_{ij} = 1$ nếu có cạnh nối đỉnh i với đỉnh j
 $= 0$ nếu ngược lại
- Mảng: $Daqua[1..n]$ đánh dấu đỉnh i đã được đi qua trên đường đi hay chưa? $Daqua[i] = true$ nếu đỉnh i đã có trên đường đi
 $= false$ nếu ngược lại, đỉnh i chưa có trên đường đi
Khởi tạo: $Daqua[1..n] = false$

§ 2. Phương pháp quay lui

- **Ví dụ 2: Tìm đường trong mê cung**

- **Lược đồ:**

Backtrack(i) $\equiv$

```
    for (v=1..n)    //duyet qua các đỉnh
    |
    |    if ((m[xi-1,v]=1) & (not Daqua[v])) //v chấp nhận được
    |    |
    |    |    xi = v;
    |    |
    |    |    Daqua[v] = true; //ghi nhận trạng thái đã chọn v
    |    |    if (xi= F) printResult (x1,x2,...,xi);
    |    |    else Backtrack(i+1) ;
    |    |
    |    |    Daqua[v] = false; //khôi phục trạng thái chưa chọn v
    |    |
    |    endif;
    endfor;
```

End.

Lời gọi ban đầu: x₁=S;

Backtrack(2);

§ 2. Phương pháp quay lui

- **Các bài toán khác**

- Tìm chu trình Hamilton của đồ thị (Hamiltonian Path Problem)
- Bài toán người đưa hàng (Traverling Salesman Problem)
- Bài toán xếp balo (Knapsack Problem)
- Bài toán xếp quân cờ: Xếp n quân hậu/mã trên bàn cờ $n \times n$ sao cho không quân nào không chế quân nào.
-

§ 3. Phương pháp nhánh cận

- **Bài toán tối ưu**

- Bài toán: Tìm ra phương án tốt nhất thỏa mãn một số yêu cầu nào đó.
- Hai hướng tiếp cận tìm lời giải tối ưu cho bài toán:
 - Tìm từng lời giải, khi hoàn tất một lời giải thì so sánh chi phí của nó với chi phí tốt nhất hiện có. Nếu tốt hơn thì cập nhật chi phí tốt nhất mới.
 - Với mỗi lời giải, khi xây dựng các thành phần nghiệm luôn kiểm tra điều kiện nếu đi tiếp theo hướng này thì có khả năng nhận được lời giải tốt hơn lời giải hiện có không? Nếu không thì thôi không đi theo hướng này nữa => Nguyên lý **nhánh cận** (*Branch and Bound*)

§ 3. Phương pháp nhánh cận

- **Ý tưởng**

- Nhánh cận (Branch and Bound): Thuật toán tìm lời giải cho các bài toán tối ưu dạng liệt kê cấu hình dựa trên nguyên lý đánh giá nhánh cận.
- **Nguyên lý nhánh cận:** Sử dụng các thông tin đã tìm được trong lời giải của bài toán để loại bỏ sớm phương án không dẫn tới lời giải tối ưu
- Bản chất: Sử dụng phương pháp quay lui nhưng tại mỗi bước đưa thêm thao tác đánh giá chi phí của phương án hiện có.
 - Nếu đó là phương án tối ưu hoặc có hy vọng trở thành phương án tối ưu (tức là tốt hơn phương án hiện có) thì cập nhật lại phương án tối ưu và đi tiếp theo hướng đó.
 - Trong trường hợp ngược lại thì bỏ qua hướng đang xét.

§ 3. Phương pháp nhánh cận

- **Mô hình**

Tập khả năng $D=\{(x_1, x_2, \dots, x_n)\}$ gồm các cấu hình có dạng $(x_1, x_2, \dots, x_n)$.

Hàm chi phí $f: D \rightarrow Z$

Nghiệm của bài toán: $x=(x_1, x_2, \dots, x_n)$ sao cho $f(x)=$ giá trị tối ưu (max/min)

- Cho x_i nhận lần lượt các giá trị có thể. Với mỗi giá trị thử gán cho x_i xét khả năng chọn $x_{i+1}, x_{i+2} \dots$
- Tại mỗi bước i : Xây dựng thành phần x_i
 - Xác định x_i theo khả năng v .
 - Tính chi phí lời giải nhận được. Nếu “tốt hơn” lời giải hiện thời thì chấp nhận x_i theo khả năng v . Tiếp tục xác định $x_{i+1}, \dots$ đến khi gặp nghiệm
 - Nếu không có một khả năng nào chấp nhận được cho x_i hoặc lời giải xấu hơn thì lùi lại bước trước để xác định lại thành phần x_{i-1} .

§ 3. Phương pháp nhánh cận

- **Lược đồ**

$\text{Try}(i, S) \equiv$ *//Sinh thành phần thứ i của cấu hình với chi phí hiện thời S*

for (v thuộc tập khả năng của x_i)

if (v là chấp nhận được)

$T = S +$ Chi phí nghiệm khi có thêm thành phần v;

if (T tốt hơn T_{optimize}) *//Tốt hơn chi phí tốt nhất hiện có*

$x_i = v$;

 <Ghi nhận trạng thái chấp nhận v>;

if (x_i là trạng thái kết thúc)

 <Ghi nhận nghiệm>;

$T_{\text{optimize}} = T$; *//Cập nhật chi phí tốt nhất*

else

$\text{Try}(i+1, T)$; *//sinh thành phần tiếp theo với chi phí hiện thời T*

endif;

 <Khôi phục trạng thái chưa chấp nhận v>;

endif;

endif;

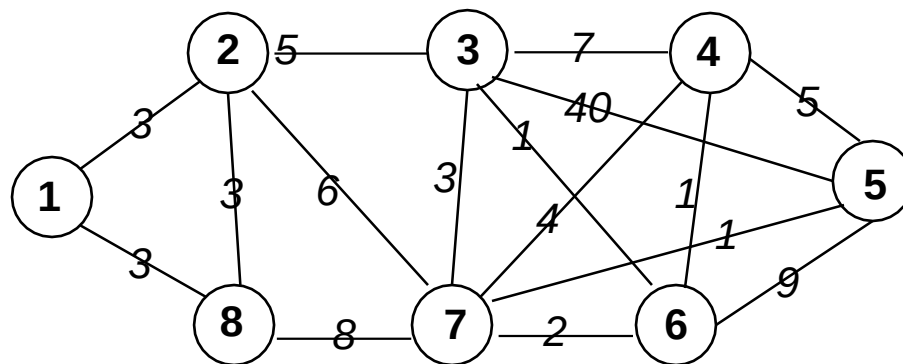
End.

§ 3. Phương pháp nhánh cận

- **Ví dụ:** Bài toán người bán hàng (*Traveling Salesman*): Một người bán hàng trên hệ thống n thành phố. Giữa các thành phố có thể có hoặc không có đường đi, mỗi đường đi có chi phí xác định từ trước.

Người bán hàng xuất phát từ một thành phố, đi tới tất cả các thành phố khác đúng một lần và quay lại thành phố ban đầu.

Hãy xác định một hành trình sao cho tổng chi phí nhỏ nhất.



§ 3. Phương pháp nhánh cận

- **Ví dụ: Người bán hàng**

- Phân tích: Biểu diễn mạng lưới giao thông giữa các thành phố bởi một đồ thị *có trọng số* $G=(V,E)$
 - Mỗi thành phố là một nút của đồ thị
 - Mỗi đường đi giữa các thành phố là một cạnh nối giữa các nút của đồ thị, trên đó có ghi trọng số là chi phí đường đi.
Các cặp nút không có đường đi: trọng số là ∞
 - Đỉnh xuất phát $\equiv$ kết thúc: $S \in V$
 - Đường đi tìm được là dãy $S=x_1,x_2,\dots,x_n,x_1=S$ với $x_i \in V, (x_i,x_{i+1}) \in E$, có tổng chi phí nhỏ nhất
- $\Rightarrow$ Sinh các dãy hoán vị $1..n$ và tính dãy có chi phí nhỏ nhất:
- Quay lui
 - Nhánh cận

§ 3. Phương pháp nhánh cận

- **Ví dụ: Người bán hàng**

- Phân tích: Tiếp cận theo phương pháp nhánh cận

- Chi phí tốt nhất đã tìm được (*BestCost*). Ban đầu $BestCost = +\infty$

- Tại mỗi bước chọn x_i : Chi phí đường đi từ x_1 đến x_{i-1} là C

- ✳ Với mỗi khả năng v , tính lại chi phí $C = C +$ chi phí từ x_{i-1} tới v

- ✳ Nếu C lớn hơn *BestCost* hoặc **không có khả năng nào chấp nhận được** cho x_i thì lùi lại bước trước để xác định lại thành phần x_{i-1} .

- Ngược lại, chấp nhận $x_i = v$. Tiếp tục xác định $x_{i+1}, \dots$

- ✳ Đến khi gặp nghiệm ($i=n+1$ & $x_i=S$):

- Cập nhật đường đi tốt nhất hiện tại đã tìm được

- Cập nhật giá trị *BestCost* mới: $BestCost = C$

§ 3. Phương pháp nhánh cận

- **Ví dụ 2: Người bán hàng**

- Mô tả dữ liệu

- Đánh chỉ số các đỉnh của đồ thị từ $1 \dots n$
- Biểu diễn đồ thị G bằng ma trận kề $M = (c_{ij})$ cỡ $n \times n$
 $c_{ij} = \text{cost}$ nếu có cạnh nối đỉnh i với đỉnh j với chi phí cost
 $= \infty$ nếu không có đường đi
- Mảng: $\text{Daqua}[1..n]$ đánh dấu đỉnh i đã được đi qua trên đường đi hay chưa?
Khởi tạo: $\text{Daqua}[1..n] = \text{false}$

§ 3. Phương pháp nhánh cận

- **Ví dụ: Người bán hàng**

- Thuật toán

$\text{Try}(i, C) \equiv$ //Sinh thành phần thứ i của cấu hình với chi phí hiện thời C

for ($v = 1..n$)

if ($c[x_{i-1}, v] < \infty$) & (not Daqua[v]))

$C = C + c[x_{i-1}, v];$

$CD = C +$ chi phí thấp nhất nếu đi tiếp từ v

if ($CD < \text{BestCost}$) //Tốt hơn chi phí tốt nhất hiện có

$x_i = v ;$

$\text{Daqua}[v] = \text{true};$

if ($i = n+1$) & ($x_i = S$)

$\langle \text{Ghi nhận nghiệm } x_1, x_2 \dots x_{n+1} \rangle;$

$\text{BestCost} = C; //\text{Cập nhật chi phí tốt nhất}$

else $\text{Try}(i+1, C); //\text{sinh thành phần tiếp theo với chi phí hiện thời } C1$

endif;

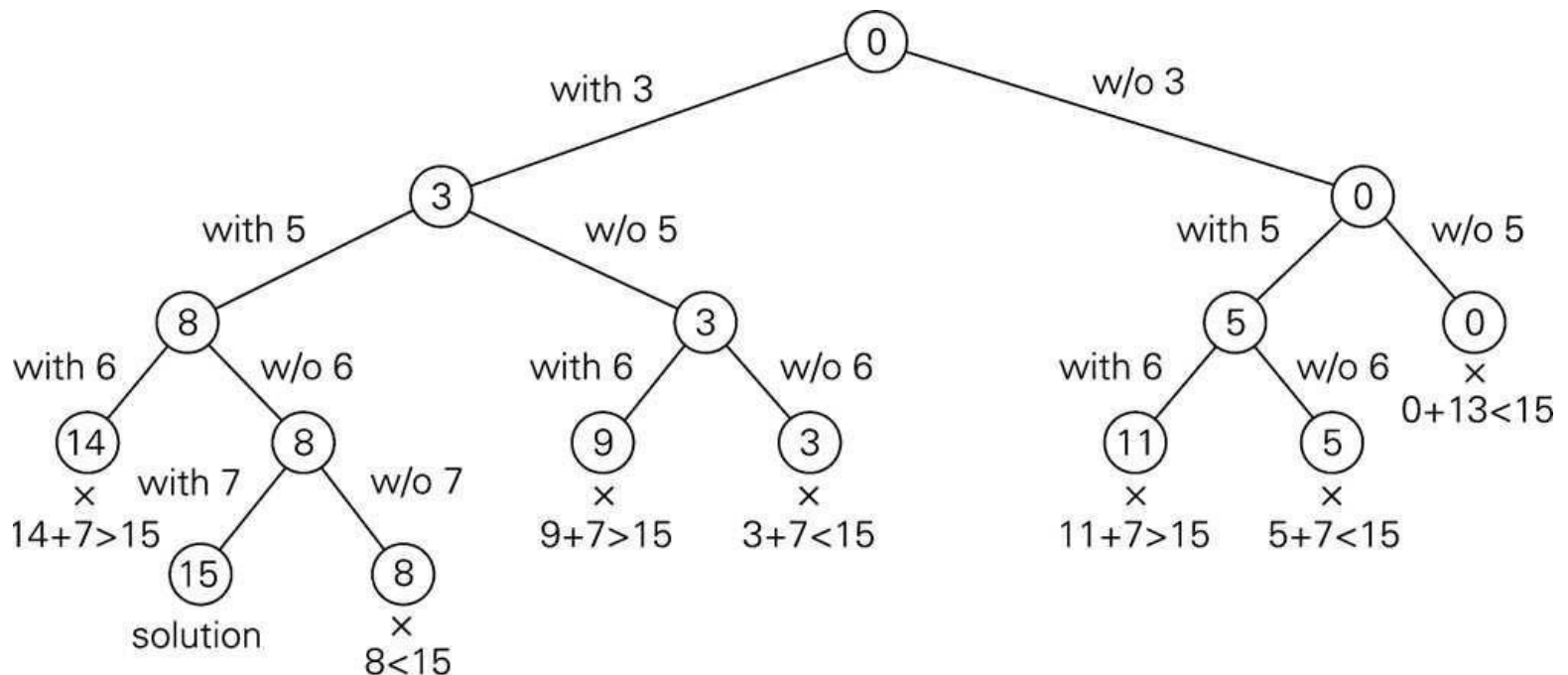
$\text{Daqua}[v] = \text{false};$

endif; endif;

End.

§ 3. Phương pháp nhánh cận

- Bài toán tổng tập con (Subset-Sum Problem):** tìm các tập con của tập $S = \{s_1, s_2, \dots, s_n\}$ biết các phần tử của tập con tổng bằng giá trị d cho trước (VD: $S = \{3, 5, 6, 7, 8, 9\}$, $d = 15$)



§ 3. Phương pháp nhánh cận

- **Các bài toán khác**

- Xếp balo (Knapsack problem)
- Người bán hàng (Traveling salesman problem - TSP)
- ...

Xem thêm http://en.wikipedia.org/wiki/Branch_and_bound

Bài tập

- Lập trình giải thuật quay lui cho các bài toán:
 - Sinh chuỗi nhị phân có n phần tử
 - Sinh chỉnh hợp chập k của n phần tử
 - Sinh tổ hợp chập k của n phần tử
 - Giải bài toán tìm nghiệm nguyên dương của phương trình tuyến tính với hệ số nguyên dương
- Lập trình giải thuật quay lui cho các bài toán:
 - Tìm đường đi trên đồ thị
 - Bài toán 8 quân hậu
 - Bài toán con mã đi tuần
- Lập trình giải thuật nhánh cận cho các bài toán:
 - Tìm đường đi với chi phí nhỏ nhất
 - Bài toán tổng tập con